My favorites ▼ | Sign in

solrmarc
Index your MaRC records with apache solr.

Search projects

Project Home Downloads Wiki Issues Source

Search Current pages for Search

ConfigProperties
Details on the config.properties file.

Updated Oct 18, 2010 by demian.k...@villanova.edu

- Introduction
- Example File
- config.properties File Entries
- Special Purpose config.properties File Entries

Introduction

The config.properties file is the top-level configuration file for controlling SolrMarc's behavior. This page describes all of the available settings. Note that the actual mapping of MARC fields to Solr fields is controlled by a separate index.properties file pointed to by this file -- see the IndexProperties page for details on mapping data.

Example File

An example config.properties file is below (more details about the entries follow):

```
# Properties for the !SolrMarc import program
#  for more documentation, see
#  http://code.google.com/p/solrmarc/wiki/ConfiguringSolrMarc

# solrmarc.solr.war.path - must point to either a war file for the version of Solr
# that you want to use, or to a directory of jar files extracted from a Solr war
# files.  If this is not provided, SolrMarc can only work by communicating with
# a running Solr server.
solrmarc.solr.war.path=jetty/webapps/solr.war

# solrmarc.custom.jar.path - Jar containing custom java code to use in indexing.
# If solr.indexer below is defined (other than the default of org.solrmarc.index.SolrIndexer)
# you MUST define this value to be the Jar containing the class listed there.
solrmarc.custom.jar.path=

# - solr.indexer.properties - indicates how to populate Solr index fields from
#   marc data.  This is the core configuration file for solrmarc.
solr.indexer.properties = demo_index.properties, demo_local_index.properties

# - solr.indexer - full name of java class with custom indexing functions. This
#   class must extend org.solrmarc.index.SolrIndexer, which is default
solr.indexer = org.solrmarc.index.SolrIndexer

# - marc_permissive - if true, try to recover from errors, including records
#  with errors, when possible
marc_permissive = true

# --- Solr instance properties ---------------------------------------------
# - solr.path - path to your Solr instance
solr.path = jetty/solr

# - solr.data.dir - path to data directory for your Solr instance
#   (note: Solr can be configured to use a different data directory)
solr.data.dir = jetty/solr/data

# - solr.hosturl - optional URL to send commit to Solr after indexing to avoid
#   need for Solr restart.  Expects an XmlUpdateRequestHandler;  see
#  http://wiki.apache.org/solr/UpdateXmlMessages
solr.hosturl = http://localhost:8983/solr/update


# -- MARC data properties ------------------------------------------------

# - marc.default_encoding - possible values are MARC8, UTF-8, UNIMARC, BESTGUESS
marc.default_encoding = MARC8

# - marc.to_utf_8 - if true, this will convert records in our import file from
#   MARC8 encoding into UTF-8 encoding on output to index
marc.to_utf_8 = true

# - marc.include_errors - when error in marc record, dump description of error
#   to field in solr index an alternative way to trap the indexing error
#   messages that are logged during index time.  Nice for staff b/c they can
#   search for errors and see ckey and record fields in discovery portal.  This
#   field is NOT used for other queries.  Solr schema.xml must have field
#   marc_error.
marc.include_errors = false
```

config.properties File Entries

- **solrmarc.solr.war.path:** Must point to either a war file for the version of Solr that you want to use, or to a directory of jar files extracted from a Solr war file. If this is not provided, SolrMarc can only work by communicating over HTTP with a running Solr server.
- **solrmarc.custom.jar.path:** Name of Jar containing custom java code to use in indexing. If solr.indexer below is defined (other than the default value of !org.solrmarc.index.SolrIndexer) you MUST define this value to be the Jar containing the class listed there.
- **solr.path:** lists the full path to the Solr home directory, which should contain the directories `etc`, `conf` and `data`. The directory `conf` should contain a file named *schema.xml* which describes the different indexing fields that the instance of the Solr search engine is expecting, and the data directory is where the Solr index files will be placed. Note that if the special value of `REMOTE` is used here, SolrMarc will use the value for solr.hosturl (described below) and send updates directly to that Solr server over a HTTP connection.
- **solr.data.dir:** Path to data directory for your Solr instance. Note that if this property is not defined, a default value of ${solr.path}/data will be used. Note also that if you are pointing at a multi-core solr configuration this value will likely be ignored, and the the data directory will be defined relative to the core specified in the property solr.core.name
- **solr.core.name:** (optional) Name of solr core to use when you are pointing at a multi-core solr configuration. This property should only be used when you are working with a multi-core solr configuration.
- **solr.indexer.properties:** lists the name of the properties file that defines the mapping from the MARC record fields and sub-fields to the Solr index entries. For details of how this mapping is done, please see the IndexProperties page. Note that multiple property files can be listed here separated by commas. Index specification entries found in the second (or subsequent) file, either add to the Index specification of the first file, or override entries found there.
- **solr.indexer:** (optional); specifies the name of a custom indexing class which defines any custom routines for extracting and mapping data for a particular Solr index field. These custom routines will only be needed in cases where values from several different MARC fields and sub-fields need to be consulted to determine the value to add to the Solr index field. Several examples of these custom indexing routines are given below. If, however, you can handle all of the necessary MARC field-to-Solr index field mappings using the available specification language, you do not need to define a value for this entry, and the default SolrIndexer class will be used.
- **solr.hosturl:** (optional); Setting this value means a "commit" message is sent to a running Solr URL when the indexing process completes. If Solr is running while its index is being updated, it doesn't automatically know about changes to the index: either Solr will need to be restarted to find the new data, or solr.hosturl can be set so that a "commit" message is sent to Solr to tell it to reload the index. The solr.hosturl value should be the URL of an XmlUpdateRequestHandler for the Solr instance, generally http://yer_solr_host:port/solr/update. To restate: if this value is given, when the indexing process completes, SolrMarc will send a signal to the currently running Solr search engine, which will cause it to read the newly modified index files. This is primarily a convenience function, to avoid having to stop and restart the running Solr search engine. If no search engine is running at the URL specified, the program will quietly proceed along its way, with the understanding that when that search engine is eventually restarted it will read in the new index data anyway.
- **marc.to_utf_8:** a boolean value, defaulting to false. If it is set to true, then as MARC records are read in the program translate the fields of the record to UTF-8, prior to sending the record to the indexer. If the records are determined to already be encoded in UTF-8 the field values will be unchanged.

  Note: You are strongly recommended to use this setting to translate your records to UTF-8, any of the other character encodings that are commonly used in MARC records likely to cause display problems for special characters that occur in the data.
- **marc.permissive:** a boolean value, defaulting to false. If it is set to true, then as MARC records are read in, if the program encounters ill-formed records, it will make an attempt to work around the problem and read in and index the record rather than simply skipping over the bad record. Note that the feature does not and cannot fix records that have a valid MARC record structure, but which have incorrect or invalid data entered in some fields.
- **marc.default_encoding:** valid values: `MARC8`, `UTF8`, `UNIMARC`, `ISO8859_1`, or `BESTGUESS`. It is used to specify what encoding is used in the marc records that you will be importing. In all likelihood your data will be in one of the first two listed above, or maybe in the third if you are a European institution. If you have no idea what encoding your original data is in, you can specify `BESTGUESS` and the program will do its best to correctly determine what encoding is used. Note also if you provide an incorrect value for this entry, *and* if you set the entry `marc.permissive` to true, the MARC record reader will try to detect when you are wrong, and do its best to recover from the problem.
- **marc.include_errors:** a boolean value, defaulting to false. If it is set, *and* if `marc.permissive` is also set to true, then any errors that are encountered in the process of reading a MARC record, will be stored in the solr index record that is produced, this will allow catalogers to review the errors encountered during reading and to correct the original records in which the errors occurred.
- **marc.unicode_normalize:** (added in SolrMarc 2.1.1) If it is set, Unicode characters found in the record will be normalized using the specified method: `C` (Canonical decomposition followed by canonical composition), `D` (Canonical decomposition), `KC` (Compatibility decomposition followed by canonical composition) or `KD` (Compatibility decomposition). More information on Unicode normalization can be found in this technical report. Some helpful commentary on the use of this setting can be found in Jonathan Rochkind's blog.

Special Purpose config.properties File Entries

Lastly, there are several special purpose entries that will not be used in general, and usually won't appear in the config.properties file.

- **marc.override:** lists the class name of the implementation of the MarcFactory object to use to override the default one provided in the `marc4j` library. You almost certainly will not need to use this entry. It is needed at UVa since the MARC records that we are working from have multiple 001 fields (which actually makes them invalid MARC records) and rather than simply keeping the last of these 001 fields that occurs (and discarding the rest), we need to specifically select the correct 001 field to use in the indexing process. Another option for this field is if you want to ensure that the fields in the MARC record are not sorted into ascending order as they are read in (which marc4j does by default) you can override this behavior by defining this property to the value `org.solrmarc.marcoverride.UVAMarcFactoryImpl`
- **solr.optimize_at_end:** a boolean value, defaulting to false. If it is set to true, when the program has finished indexing all of the MARC records provided to it, it will optimize the index, which will make searches perform much faster. This entry is set by the shell script `optimizesolr`, which is found in the dist directory.
- **marc.ids_to_delete:** supplies the name of a file that contains a list of Solr item ids, one per line, that are to be removed from the Solr index.
- **marc.delete_record_id_mapper:** used in conjunction with `marc.ids_to_delete` when the entries that occur in the `marc.ids_to_delete` file need to be processed to correspond to the actual ids that occur in the solr index. For instance at UVa the records have ids like *u184783* but the entry that would appear in the `marc.ids_to_delete` file produced by our system would be merely 184783.
- **marc.verbose:** a boolean value. If it is set to true, instructs the program to print out the entire MARC record as it is read in, followed by the entire index record that will be sent to Solr.

- **marc.include_if_present** and **marc.include_if_missing:** used for reprocessing records that have already been indexed, if a new index field is added. They are only useful when the newly added field draws its information from fields that only occur in a small subset of records. For example, in the Indexer configuration file shown below, the following field specification:

  ```
  instrument_facet = 048m[0-1], instrument_map
  ```

  specifies that the information should be found in the 048m field of the MARC record. This was a typo, it should have read 048a, therefore no index records were created with a value for `instrument_facet`. To fix it, rather than re-index all of our records, I was able to set `marc.include_if_present = 048a` which specifies that only those records that have one or more 048a fields will be re-indexed, the rest will be skipped (which is fine since the indexed record would be the same in any case.)
  Note: Rather than setting these entries, you should use the `filterrecords` script which internally uses these settings to operate as a marc -based-grep program.

- **marc.combine_records:** if a bib record with all of its holdings stuffed into a bib field (e.g. 999) is larger than the maximum allowed size of a marc21 record, then some ILS (Sirsi, for example) will write the record out multiple times, putting sequential batches of holdings in the (999) fields of each record instance. So each of these records has all of the bibliographic information, but different holdings in the (999) field. When this property is set, the "MarcCombiningReader" class is used to create a single marc record with all of the occurrences of the given field. That is, if the Sirsi export created 3 bib records due to lots of items, and the item info is in a 999 bib field, then setting marc.combined_record = 999 will create a single marc record with ALL the 999 fields for importing into Solr. If you need to combine multiple fields, you may use a regular expression.

- **marc.combine_records.left_field/marc.combine_records.right_field:** If marc.combine_records is turned on, these settings may be defined to determine which control fields are used to combine records. If these settings are omitted, records with matching control numbers will be merged. One example of using these fields: Voyager is capable of exporting both bibliographic and holdings records in a single file. The bibiographic records have unique identifiers in 001, and the holdings records refer to their "parent" bib record with the 004 field. If you want to copy the holdings information into the bibliographic records, you can set marc.combine_records.left_field to "001" and marc.combine_records.right_field to "004."

► Sign in to add a comment

©2011 Google - Terms - Privacy - Project Hosting Help

Powered by Google Project Hosting